

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №2
по курсу «Программирование графических процессоров»

Работа с матрицами. Метод Гаусса.

Выполнил: *Серый Никита Олегович*

Группа: *M8O-409B-22*

Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

1. Цель работы: Использование объединения запросов к глобальной памяти. Реализация метода Гаусса с выбором главного элемента по столбцу. Ознакомление с библиотекой алгоритмов для параллельных расчетов Thrust. Использование двухмерной сетки потоков. Исследование производительности программы с помощью утилиты nvprof (обязательно отразить в отчете). В качестве вещественного типа данных необходимо использовать тип данных double. Библиотеку Thrust использовать только для поиска максимального элемента на каждой итерации алгоритма. Все результаты выводить с относительной точностью 10^{-10} .
2. Вариант 3. Решение квадратной СЛАУ. Необходимо решить систему уравнений $Ax = b$, где A -- квадратная матрица $n \times n$, b -- вектор-столбец свободных коэффициентов длиной n , x -- вектор неизвестных.

Входные данные. На первой строке задано число n -- размер матрицы. В следующих n строках, записано по n вещественных чисел -- элементы матрицы. Далее записываются n элементов вектора свободных коэффициентов. $n \leq 10^4$.

Выходные данные. Необходимо вывести n значений, являющиеся элементами вектора неизвестных x .

Пример:

Входной файл	Выходной файл
2 1 2 3 4 5 6	-4.0000000000e+00 4.5000000000e+00

Программное и аппаратное обеспечение

Графический процессор (Google Colab)

Compute capability	7.5
Name	Tesla T4
Total Global Memory	15828320256
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)

Total constant memory	6553
Multiprocessors count	40

Процессор AMD Ryzen 5 5500U (2.1 GHz, max 4 GHz)

Technology	7 нм
Cores	6
Threads	12
Cache (Level 2)	64К (на ядро)
Cache (Level 2)	512К (на ядро)
Cache (Level 3)	8 МБ (всего)

Оперативная память

Number of modules	2
Module Size	8 ГБ
Frequency	3200 МГц
Generation	DDR4

Жесткий диск - 512 ГБ SSD

OS - Ubuntu 22

IDE - VS Code, theme Monokai; gedit, theme Oblivion

Compiler - nvcc

Метод решения

В составе алгоритма Гаусса прямой и обратный ход. На прямом ходе исходная матрица приводится к верхнетреугольному виду, где на каждом шаге совершаются три действия:

1. Поиск в текущем столбце максимального по модулю элемента.
2. Перестановка текущей строки со строкой-максимумом.
3. Вычитание текущей строки из всех нижележащих строк с определенным множителем для зануления поддиагональных элементов в столбце

На обратном ходе решается треугольная система, начиная с последней строки.

Описание программы

1. Основные типы данных

thrust::device_ptr<double>: Он позволяет `max_element` понимать, что данные находятся в памяти видеокарты, а не в оперативной памяти.

dim3: Трехмерная структура CUDA, задающая размеры сетки и блоков для конфигурации параллелизма.

2. Реализованные функции

find_pivot_row_index: Для поиска максимального элемента в столбце и его исключения преобразует сырой указатель в `device_ptr`.

solve_back_substitution: Реализует обратный ход метода Гаусса на CPU.

3. Описание ядер

swap_rows_kernel

Переставляет строки, подготавливая матрицу к текущему шагу исключения. Каждый поток отвечает за один столбец матрицы. Если текущий шаг не совпадает с индексом строки, в которой содержится максимум, поток меняет элементы в этих строках местами.

Поскольку перестановка строк линейна, используется одномерная сетка потоков. Поток с индексом 0 дополнительно меняет элементы в векторе свободных членов `b`.

```
__global__ void swap_rows_kernel(double* matrix, double* vector_b, int n, int row_a, int row_b) {
```

```
    for (int col_idx = blockIdx.x * blockDim.x + threadIdx.x; col_idx < n; col_idx += blockDim.x * gridDim.x) {
```

```
        double temp_val = matrix[col_idx * n + row_a];
```

```
        matrix[col_idx * n + row_a] = matrix[col_idx * n + row_b];
```

```
        matrix[col_idx * n + row_b] = temp_val;
```

```
        if (col_idx == 0) {
```

```
            double temp_b = vector_b[row_a];
```

```
            vector_b[row_a] = vector_b[row_b];
```

```
            vector_b[row_b] = temp_b;
```

```
    }
```

```
}  
  
}
```

forward_elimination_kernel

Выполняет прямой ход метода Гаусса, зануляя элементы под диагональю и пересчитывая коэффициенты в оставшейся части матрицы.

pivot_value - текущий диагональный элемент, на который происходит деление.

multiplier - коэффициент умножения опорной строки перед вычитанием из текущей.

```
__global__ void forward_elimination_kernel(double* matrix, double* vector_b, int n, int  
step_idx) {  
  
    for (int target_row = blockIdx.x * blockDim.x + threadIdx.x + step_idx + 1; target_row <  
n; target_row += blockDim.x * gridDim.x) {  
  
        double pivot_value = matrix[step_idx * n + step_idx];  
  
        double multiplier = matrix[step_idx * n + target_row] / pivot_value;  
  
        for (int current_col = blockIdx.y * blockDim.y + threadIdx.y + step_idx + 1; current_col  
< n; current_col += blockDim.y * gridDim.y) {  
  
            matrix[current_col * n + target_row] -= multiplier * matrix[current_col * n +  
step_idx];  
  
        }  
  
        if (blockIdx.y * blockDim.y + threadIdx.y == 0) {  
  
            vector_b[target_row] -= multiplier * vector_b[step_idx];  
  
        }  
  
    }  
  
}
```

Результаты

1. Отобразить в виде таблички или графиков замеры времени работы ядер с различными конфигурациями (начиная с <<< 1, 32 >>> и как минимум до <<< 1024, 1024 >>>, для ЛР с MPI с различным числом процессов) и различными входными данными (небольшие тесты, средние и предельные).
Обязательно указать единицы измерения времени.

Единицы измерения времени в сравнениях - ms.

1. Конфигурация <<< 1, 32 >>>

n	time (ms)
2	1.0301
512	493.9895, 465.4126, 520.5010
1024	3795.9084, 3807.8726, 3836.4795

2. Конфигурация <<< 256, 256 >>>

n	time (ms)
2	1.0246
512	49.0232, 60.4090, 49.0268
1024	137.4235, 125.6496, 124.8629

3. Конфигурация <<< 512, 512 >>>

n	time (ms)
2	1.0578
512	58.2656, 49.8136, 55.9865
1024	129.2796, 128.4383, 127.6498

4. Конфигурация <<< 1024, 1024 >>>

n	time (ms)
2	1.0338
512	83.2803, 83.0259, 82.8314
1024	214.9117, 223.6744, 220.1001

2. Произвести сравнение с CPU (для этого нужно реализовать свой вариант ЛР без использования технологии CUDA / OpenMP).

```
l-4 > src > G nocuda.cpp > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <chrono>
5
6  bool absolute_compare(double a, double b) {
7      return fabs(a) < fabs(b);
8  }
9
10 void swap_rows_cpu(double* matrix, double* vector_b, int n, int row_a, int row_b) {
11     for (int col_idx = 0; col_idx < n; col_idx++) {
12         double temp_val = matrix[col_idx * n + row_a];
13         matrix[col_idx * n + row_a] = matrix[col_idx * n + row_b];
14         matrix[col_idx * n + row_b] = temp_val;
15     }
16     double temp_b = vector_b[row_a];
17     vector_b[row_a] = vector_b[row_b];
18     vector_b[row_b] = temp_b;
19 }
20
21 void forward_elimination_cpu(double* matrix, double* vector_b, int n, int step_idx) {
22     for (int target_row = step_idx + 1; target_row < n; target_row++) {
23         double pivot_value = matrix[step_idx * n + step_idx];
24         double multiplier = matrix[step_idx * n + target_row] / pivot_value;
25
26         for (int current_col = step_idx + 1; current_col < n; current_col++) {
27             matrix[current_col * n + target_row] -= multiplier * matrix[current_col * n + step_idx];
28         }
29         vector_b[target_row] -= multiplier * vector_b[step_idx];
30     }
31 }
32
33 int find_pivot_row_index(double* matrix, int n, int step_idx) {
34     int best_row = step_idx;
35     for (int i = step_idx + 1; i < n; i++) {
36         if (absolute_compare(matrix[step_idx * n + best_row], matrix[step_idx * n + i])) {
37             best_row = i;
38         }
39     }
40     return best_row;
41 }
42
43 void solve_back_substitution(double* matrix, double* vector_b, double* result, int n) {
44     for (int i = n - 1; i >= 0; --i) {
45         double sum = 0;
46         for (int j = i + 1; j < n; ++j) {
47             sum += matrix[j * n + i] * result[j];
48         }
49     }
50 }
```

Рис. 1 — Реализация ЛР без использования технологии CUDA.

```
snowwy@snowwy-BOM-WXX9:~/Documents/pc
2
1 2 3 4 5 6
CPU Total Time: 0.0024 ms
-4.0000000000e+00 4.5000000000e+00
```

Рис. 2 — Тест на 2 уравнения.

```
17 C++/src/gen.py
Файл input.txt на 512 уравнений успешно создан.
snowwy@snowwy-BOM-WXX9:~/Documents/poor_monk/poc
CPU Total Time: 223.7894 ms
snowwy@snowwy-BOM-WXX9:~/Documents/poor_monk/poc
```

Рис. 3 — Тест на 512 уравнений.

```
snowwy@snowwy-BOM-WXX9:~/Documents/poor_monk/poc
CPU Total Time: 3990.9486 ms
snowwy@snowwy-BOM-WXX9:~/Documents/poor_monk/poc
```

Рис. 4 — Тест на 1024 уравнения.

```
[55] ✓ 10s %%shell
      nvcc -arch=sm_75 code.cu
      ./a.out
      2
      1 2 3 4 5 6
      -4.000000000000e+00 4.500000000000e+00
```

Рис. 5 — Запуск работы в Google Collab.

Выводы

Сложность программирования параллельной версии метода Гаусса на CUDA выше по сравнению с CPU-версией. Основные проблемы в реализации связаны с завершением вычислений потоков. Кроме того, для редукции при поиске максимума в столбце на GPU пришлось использовать отдельную библиотеку Thrust. Адаптировать гетерогенную программу под различные конфигурации сетки стало возможным благодаря `dim3`, но усложнена логика циклов внутри ядер за счет индексов. В конфигурации 1, 32 параллелизм GPU простаивает, а накладные расходы на запуск ядер и передачу данных остаются, поэтому результаты замеров довольно медленные. При достижении определенных параметров время перестает падать. Это означает, что алгоритм уперся в пропускную способность памяти.